Lecture 1
First Steps: The Terminal, Configuration, and Your First Repository

Moving around in a terminal, telling Git who you are, `git init`, and `git status`

Learning goals

- Move around the file system and create files from a terminal.
- Tell Git who you are, and understand the three places where settings live.
- Turn an ordinary folder into a Git repository with `git init`.
- Explain what the hidden `.git` folder is, and why you must not delete it.
- Read the output of `git status`.
- Find help for any Git command on your own.

1 The terminal in ten minutes

Git is used mostly through a **terminal**: a window where you type a command, press Enter, and the computer replies with text. Point-and-click programs for Git exist, but the terminal is the same on every computer, it is what every tutorial and every colleague uses, and it shows you exactly what Git is doing. A little practice here pays off in every later lecture.

The program inside the terminal that reads your commands is called a **shell**. With Git Bash on Windows, and on Linux, it is usually `bash`; on macOS it is `zsh`. (Windows also has PowerShell, which differs: use Git Bash in this course.) The commands in this lecture work the same in all of them.

History

The Unix shell dates from 1971. Bash ("Bourne-again shell") was first released in 1989 as part of the GNU project and became the default on most Linux systems. Git was designed on Linux, which is why its commands feel like shell commands.

1.1 The prompt and your location

When the terminal is ready for a command it shows a **prompt**. It usually ends with `$` or `%`. Its exact look differs between computers, so in this course we write just `$`. You do *not* type the `$`.

At every moment the terminal is "standing" in one folder, called the **current directory** or **working directory**. Almost every command acts on the folder you are standing in, so always know where you are.

```
$ pwd
/Users/sam
$ ls
Desktop    Documents    Downloads
```

`pwd` ("print working directory") prints where you are. `ls` ("list") shows what is inside. Your output will differ.

1.2 Moving around

Folders form a tree: every folder sits inside another folder, up to the top (the *root*).

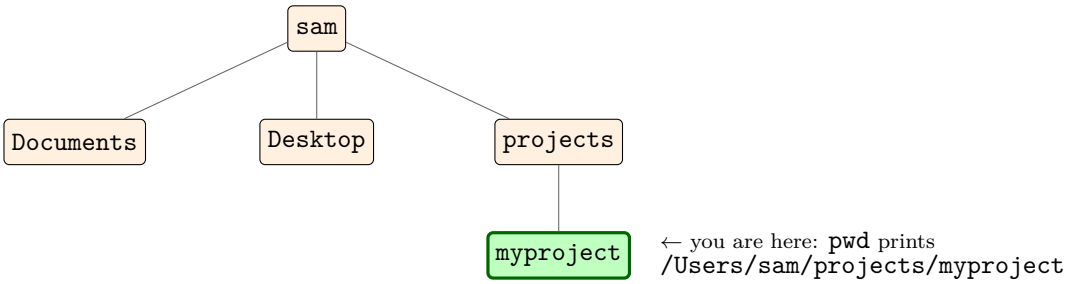


Figure 1: A folder tree (a home folder named `sam`). The green folder is the current directory.

Command	What it does
`pwd`	Print the current directory.
`ls`	List what is in the current directory. `ls -a` also shows *hidden* files (names starting with a dot).
`cd projects`	Change directory: go into the folder `projects`.
`cd ..`	Go up one level (`..` means "the parent folder").
`cd` or `cd ~`	Go back to your home folder (`~` is shorthand for it).
`mkdir name`	Make a new folder.
`echo "text" > file`	Write the text into a file. The symbol `>` sends a command's output into a file instead of the screen, creating the file or *replacing* it; `>>` *appends* instead.
`cat file`	Print the contents of a file.
`rm file`	Delete a file *permanently* (there is no recycle bin). `rm -r folder` deletes a folder and everything in it. Use with care.

```
$ cd projects            # go into projects
$ mkdir myproject        # make a new folder
$ cd myproject           # go into it
$ pwd
/Users/sam/projects/myproject
$ echo "Hello Git" > hello.txt
$ ls
hello.txt
$ cat hello.txt
Hello Git
$ cd ..                  # back up to projects
```

Key idea

Press the **Tab** key to auto-complete file and folder names, and the **Up arrow** to bring back your previous command. Both save a lot of typing and a lot of typos.

Pitfall

Spaces separate the parts of a command, so a folder name with a space must be put in quotes (`cd "my project"`). To avoid this, use names without spaces, such as `my-project`. Also type names exactly: Linux is case sensitive (`Hello.txt` and `hello.txt` are different files), while macOS and Windows usually ignore case but keep it.

Try it

Open your terminal and, without looking above, do this: print where you are, list the files, go into your `Documents` folder, make a folder called `terminal-play`, go into it, create a file `a.txt` containing the word `hi`, print the file, go back up one level, and finally clean up with `rm -r terminal-play`.

2 Telling Git who you are

Every saved version in Git records *who* made it. Git does not guess: you must give it your name and email once. This is the step you already did in Homework 0. Here we see what it really did.

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
```

The pattern is `git config <scope> <setting> <value>`. Settings have a dotted name such as `user.name`. The word `--global` says *where* to store the setting. There are three places, called **scopes**:

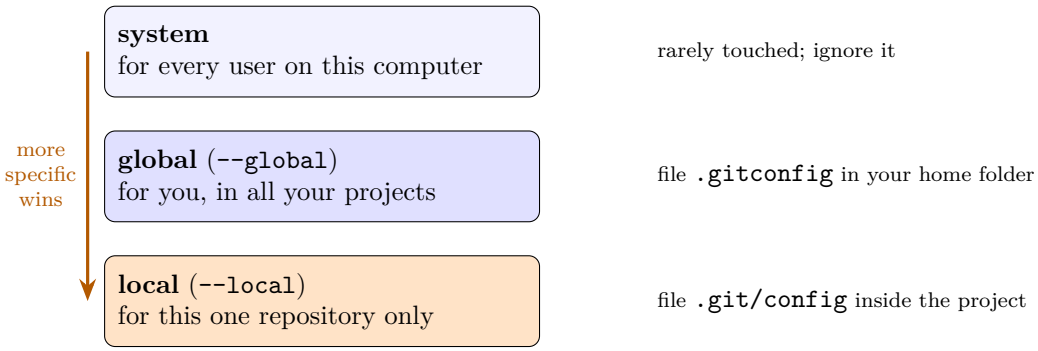


Figure 2: Where settings live. If the same setting appears in several places, the most specific one wins.

The usual use of this: your global email is your personal one, but for one particular project (say, for work) you want a different email. Setting it *inside that repository* overrides the global value only there.

To see all your settings, or just one, and *where* each comes from:

```
$ git config --list --show-origin
file:/home/sam/.gitconfig       user.name=Your Name
file:/home/sam/.gitconfig       user.email=you@example.com
file:/home/sam/.gitconfig       init.defaultbranch=main
$ git config user.name
Your Name
```

(Your list will have more lines. Git also lowercases setting names when it lists them.) The `init.defaultbranch` line is the setting from Homework 0.3: the name given to the first branch of every new repository.

Inside a repository, `git config` with no scope sets the *local* value, which then wins over the global one:

```
$ git config user.email "work@example.com"    # local (inside a repository)
$ git config user.email                       # the winner is shown
work@example.com
$ git config --global user.email              # global value is untouched
you@example.com
```

Pitfall

The email in your Git settings is written into every version you save, and becomes public if you push to a public repository. If you do not want your real address public, GitHub offers a private "noreply" address in its settings. We return to this in Lecture 6.

3 Creating a repository

A **repository** (or *repo*) is a project folder that Git is keeping track of. Turning an ordinary folder into a repository takes one command, run from inside the folder:

```
$ cd ~/projects/myproject
$ git init
Initialized empty Git repository in /Users/sam/projects/myproject/.git/
$ ls
hello.txt
$ ls -a
.    ..    .git    hello.txt
```

Nothing visible changed in your files, but a new hidden folder, `.git`, has appeared. (Output of `ls -a` also lists `.` and `..`: these mean "this folder" and "the parent folder".)

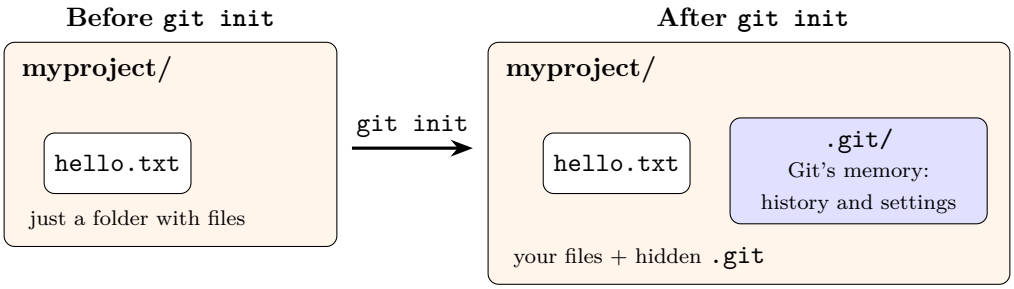


Figure 3: A repository is a normal folder plus a hidden `.git` folder. Your own files are untouched.

Key idea

A repository is *not* a special kind of file or a database somewhere else. It is a folder that contains a `.git` subfolder. The `.git` folder holds the whole history of the project and the repository's own settings. Delete `.git` and the folder is an ordinary folder again, with its history gone.

You can look inside, but do not edit it by hand:

```
$ ls .git
HEAD    config    description    hooks    info    objects    refs
$ cat .git/HEAD
ref: refs/heads/main
```

(You may see a few more entries; it depends on your Git.) You do not need to understand these yet. Lecture 10 opens this folder properly. For now notice two things: `config` is the file for the *local* settings of Figure 2, and `HEAD` names the branch you are on (here `main`, the name you set in Homework 0; Lectures 3 and 4 explain `HEAD`).

Quick fact

If your new repository does not start on `main` (for example you skipped Homework 0.3), create it with `git init -b main`, or rename the branch afterwards with `git branch -m main`. Check the current branch name any time with `git status`.

Pitfall

Run `git init` once per project, and only inside the project folder. Never run it in your home folder or your Desktop: Git would try to track everything there. Also do not create a repository inside another repository; the inner one confuses the outer one. If you are unsure where you are, run `pwd` first. (In this course the course folder is itself a repository, and `practice/` lives inside it. That is why you must `git init` inside each practice folder, so that it becomes its own, separate repository.)

4 Asking Git what is going on: `git status`

The command you will type most often is `git status`. It tells you which branch you are on and what changed since the last saved version. Try it in your new repository, first with only the file `hello.txt` from before:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Read it line by line:

`On branch main`
You are on the branch called `main`. (Branches are Lecture 4.)

`No commits yet`
You have not saved any version. A *commit* is one saved version of the project. You make the first one in Lecture 2.

`Untracked files`
Git can see `hello.txt` in the folder, but it is not tracking it: it is not part of any saved version. The line in brackets tells you the command to use next, `git add`. Git's messages often tell you the next step, so read them.

In an empty repository the message ends with `nothing to commit (create/copy files and use "git add" to track)`.

Try it

Create a new folder, run `git init` in it, and run `git status`. Now add a file with `echo "x" > a.txt` and run `git status` again. What changed in the output? Then create a second file and look again.

Key idea

`git status` never changes your files or your history. It is always safe, so run it whenever you are unsure. A common habit: run it before and after every other Git command.

5 Getting help

No one remembers all of Git. Two ways to look things up from the terminal:

```
$ git config -h          # short summary of options
$ git help config        # full manual page (same as: git config --help)
```

The full manual opens in a viewer: press the arrow keys or space to scroll and `q` to quit. On Windows it opens in your web browser. The same manuals are online at https://git-scm.com/docs.

Error messages are also help: read them from the top. Git usually says what is wrong and often what to do.

Your workflow so far: open a terminal, `cd` to the project, `git init` (once only), create or edit files, `git status`, and repeat. In Lecture 2 a "save a version" step joins the loop.

6 Homework 1

Homework 1: Your first repository

All work happens inside the `practice/` folder of the course folder. From the course folder (`LearningGit/`), run:

```
$ bash codeHints/L01/setup_hw1.sh
```

It only prepares the `practice/` folder and prints a reminder. **You do the tasks yourself**; the setup script does not build the repository for you.

1.1 Folder
Create a folder `hw1` inside `practice/` and go into it.

1.2 Repository
Turn `hw1` into a Git repository whose first branch is called `main`. (*Warning:* `practice/` sits inside the course repository. If `git status` inside `hw1` does not say `No commits yet`, or shows a branch name other than `main`, you forgot `git init` in `hw1`.)

1.3 A file
Create a file `hello.txt` containing the single line `Hello Git`.

1.4 Local identity
Set a *local* identity inside this repository only: name `Practice User` and email `practice@example.com`. Your global settings must stay as they were.

1.5 Status
Save the output of `git status` into a file called `status.txt` (hint: `git status > status.txt`).

1.6 Question
Create a file `answer.txt` whose only content is the name of the hidden folder (its name starts with a dot) in which Git keeps a project's history.

Check yourself: from the course folder run `bash codeHints/L01/check_hw1.sh`. Aim for all `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L01/`). Do not commit anything yet: you learn how in Lecture 2.

7 Summary

- The terminal runs commands in a current directory: `pwd`, `ls`, `cd`, `mkdir`, `echo`, `cat`.
- `git config` stores settings at three scopes (system, global, local); the most specific wins.
- `git init` makes a folder a repository by creating a hidden `.git` folder that holds the history.
- `git status` is safe and read-only: run it often and read its messages. `git help <command>` shows the manual.

Further reading: *Pro Git* (https://git-scm.com/book), Chapters 1.6 and 2.1; manual pages `git help init`, `config`, `status`.

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 1

Run these from the course folder `LearningGit/`. Your prompt and the exact path will differ.

```
$ cd practice
$ mkdir hw1
$ cd hw1
$ git init -b main
Initialized empty Git repository in .../practice/hw1/.git/
$ echo "Hello Git" > hello.txt
$ git config --local user.name "Practice User"
$ git config --local user.email "practice@example.com"
$ git status > status.txt
$ echo ".git" > answer.txt
```

1.1. `mkdir hw1` creates the folder and `cd hw1` enters it (do this from inside `practice/`).

1.2. `git init` creates the repository. The branch name comes from the global setting you made in Homework 0; `git init -b main` states it explicitly (Git 2.28 or newer), so it is the safe choice. If you forgot and your branch has another name (for example `master`), rename it with `git branch -m main`.

1.3. `echo "Hello Git" > hello.txt` writes one line. Check with `cat hello.txt`. Note that `>` *replaces* the file each time; `>>` would append.

1.4. `--local` is the default for `git config` inside a repository, so you could omit it, but writing it makes the intent clear. Verify where the values come from, and that your global ones are unchanged:

```
$ git config --show-origin user.name
file:.git/config        Practice User
$ git config --global user.name
Your Name
```

1.5. After the redirect, `status.txt` should look like this (it lists itself as untracked too, because the file already exists when Git runs):

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt
        status.txt

nothing added to commit but untracked files present (use "git add" to track)
```

1.6. The folder is `.git`. The file may contain `.git` or `.git/`.

Common mistakes.

- Running `git init` in the wrong folder (check with `pwd`). Do *not* delete anything until you are sure: `rm -r .git` is only safe in a folder you created for this exercise and just ran `git init` in by mistake. Never run it inside `LearningGit/` itself, because that would erase the course repository's history.
- Forgetting that `git init` is needed inside `practice/hw1`. Without it, `git status` would talk about the *course* repository above it.
- Typing the `$` at the start of a command.
- Using `--global` instead of `--local` in 1.4. That changes your name for every project. Undo it by setting your real name and email again with `git config --global`.